

Python

DRY

DRY – расшифровывается как Don't Repeat Yourself – не повторяйся, также известен как DIE – Duplication Is Evil – дублирование это зло. Этот принцип заключается в том, что нужно избегать повторений одного и того же кода. Лучше использовать универсальные свойства и функции.

KISS

KISS – Keep It Simple, Stupid – не усложняй! Смысл этого принципа программирования заключается в том, что стоит делать максимально простую и понятную архитектуру, применять шаблоны проектирования и не изобретать велосипед.

YAGNI

Термин YAGNI значит You Ain't Gonna Need It – вам это не понадобится! Его суть в том, чтобы реализовать только поставленные задачи и отказаться от избыточного функционала.

SOLID

Принцип SOLID в упрощенном варианте означает, что когда при написании кода используется несколько принципов вместе, то это значительно облегчает дальнейшую поддержку и развитие программы.

1. Single responsibility principle – принцип единственной обязанности (на каждый класс должна быть возложена одна-единственная обязанность);
2. Open/closed principle – принцип открытости/закрытости (программные сущности должны быть закрыты для изменения но открыты для расширения);
3. Liskov substitution principle – принцип подстановки Барбары Лисков (функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, не зная об этом. Подклассы не могут замещать поведения базовых классов. Подтипы должны дополнять базовые типы);
4. Interface segregation principle – принцип разделения интерфейса (много специализированных интерфейсов лучше, чем один универсальный);
5. Dependency inversion principle – принцип инверсии зависимостей (зависимости внутри системы строятся на основе абстракций. Модули верхнего уровня не зависят от модулей нижнего уровня. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций);

ACID

ACID (Atomicity - атомарность, consistency - консистентность, isolation - изолированность, durability - стойкость) это стандартный набор свойств, которые гарантируют, надежность транзакции.

В частности, ACID имеет отношение к тому, как БД может восстанавливаться после ошибок, возникающих в процессе выполнения транзакции.

В базах данных, следующих принципу ACID, данные остаются целостными и консистентными, несмотря на любые ошибки.

Atomicity (атомарность)

Атомарность гарантирует, что каждый запрос в транзакции будет выполнен успешно, либо вообще никакой, в случае ошибки одного. Не получится так, что часть запросов выполнятся успешно, а часть с ошибкой. Если хоть одна часть транзакции выполнится с ошибкой, вся транзакция не выполнится. Другими словами под атомарностью можно понимать «всё или ничего».

Consistency (консистентность, согласованность)

Это свойство даёт гарантию того, что все данные будут целостны. Данные будут корректны в соответствии со всеми предопределёнными правилами, ограничениями, каскадами и триггерами, применёнными к БД.

Isolation (изолированность)

Гарантирует, что все транзакции будут выполняться изолированно. Ни одна транзакция не зааффеетит на другую транзакцию. Другими словами, одна транзакция не сможет прочесть данные второй транзакции, которая ещё не выполнилась.

Durability (стойкость)

Durability означает, что когда транзакция будет применена, она останется в системе, даже если БД упала сразу после выполнения этой транзакции. Любые изменения, внесённые транзакцией, должны оставаться навсегда. Если БД сообщила об успешном выполнении транзакции, то она должна быть действительно применена.

pydantic

Библиотека, которая помогает работать с валидацией и сериализацией.

```
pip install pydantic
```

Расширения

```
# install email-validator  
pip install pydantic[email]
```

```
# install typing-extensions  
pip install pydantic[typing_extensions]
```

```
# install python-dotenv  
pip install pydantic[dotenv]
```

```
# install all dependencies  
pip install pydantic[email,typing_extensions,dotenv]
```

```
from pydantic import BaseModel, ValidationError, validator
```

```
class UserModel(BaseModel):
```

```
    name: str
```

```
    username: str
```

```
    password1: str
```

```
    password2: str
```

```
    @validator('name')
```

```
    def name_must_contain_space(cls, v):
```

```
        if ' ' not in v:
```

```
            raise ValueError('must contain a space')
```

```
        return v.title()
```

```
    @validator('password2')
```

```
    def passwords_match(cls, v, values, **kwargs):
```

```
        if 'password1' in values and v != values['password1']:
```

```
            raise ValueError('passwords do not match')
```

```
        return v
```

```
    @validator('username')
```

```
    def username_alphanumeric(cls, v):
```

```
        assert v.isalnum(), 'must be alphanumeric'
```

```
        return v
```

Сериализация

`.parse_raw(str)`

Обратная операция

`.json()`

Работа с orm

```
from pydantic_sqlalchemy import sqlalchemy_to_pydantic
```

```
PydanticUser = sqlalchemy_to_pydantic(User)
```

```
pydantic_user = PydanticUser.from_orm(user)
```

Задача

Вынести сериализацию в отдельный файл `serializers.py`

Добавить методы для получения dict, json из объектов ORM

Резюме и поиск работы

Сложности поиска работы у программистов во многом связаны с тем, что, несмотря на востребованность профессии, идеально подходящих вакансий не всегда много, а работодатель ищет в резюме конкретные навыки, необходимые именно его компании.

Создание резюме

Только в pdf. Забудьте о .docx, забудьте о бесплатных заменителях Word'a (если только это не Google Docs, откуда вы можете безболезненно выгрузить .pdf) и их форматах. Заменители не всегда корректно формируют пдф, так что будьте аккуратны и проверяйте результат перед отправкой.

Почему pdf? Потому что если вы составите резюме в формате .pages, открыть его на винде рядовому рекрутеру будет невозможно, а раздражающие мелочи - не то, что нужно при поиске работы. А если это .docx, маководы без проблем зальют его в Google Docs (и идеальные отступы наверняка будут испорчены не полной совместимостью), но попутно обвинят вас в компьютерной безграмотности. PDF придумали ради универсальности: хоть с телефона смотри, хоть с умных часов. Пользуйтесь им, и будет вам счастье.

Сервисы создания резюме

1. LinkedIn. Самый страшный, чудовищный и неюзабельный сайт на планете, соревнующийся только с фэйсбуком своей красотой, даст вам развёрнутые примеры оформления каждой строчки в вашем резюме. Если вы готовы погуглить, как найти на сайте нужную кнопку - создавайте резюме, впоследствии оно может принести вам крутой оффер. :)

2. Headhunter и ближайшие аналоги. Здесь нет супер-советов по оформлению. Зато всё наглядно, формат привычен эйчарскому глазу, и удобно откликаться на вакансии, размещённые на hh, здесь же лежащим резюме. У него крутые настройки приватности.

3. Moikrug. Подсказок не будет вообще, а резюме лежит в открытом доступе, если только вы не решите его удалить. Зато опять же удобно откликаться на вакансии, которые некоторые компании публикуют только здесь. Ресурс исключительно айтишный, даже менеджерские вакансии тут вывешивают редко, имейте в виду.

4. Canva.com — сервис, позволяющий бесплатно создать стильное и привлекательное резюме при помощи сотен различных дизайнерских шаблонов на любой вкус. Выбираете один из них, настраиваете цвета, шрифты, фон и сам макет, а затем вносите вашу информацию и фото. Удобно и просто.

5. Cvmaker.com — бесплатный ресурс, который дает возможность создавать резюме, используя один из девяти классических шаблонов, отличающихся своей лаконичностью и отсутствием излишеств. Хороший вариант для поклонников легкого минимализма.

Опыт работы

Во-первых, маленькая подсказка, как вычлени́ть из вашего опыта достижения, и зачем:

Возьмите за правило использовать американский подход к оформлению резюме: говорите не о том, что делали, а о том, чего достигли и чему научились. Причём достижением может быть и участие в крутом проекте, главное правильно это подать.

Например:

- Открыл в себе усидчивость, граничащую со способностью прирасти к рабочему месту, пока задача не выполнена.
- Оптимизировал процесс заполнения ведомостей...
- Научил экселевским формулам и хитрым приёмам эксель-ушу трёх ведущих преподавателей.

Где взять опыт работы?

1. Open source проекты
2. Хакатоны
3. Pet проекты
4. Фриланс

Опыт работы

Самая болезненная тема: опыта или нет, или он не релевантен интересующей вакансии. Общий совет - если есть хоть какой-то опыт - лучше указать, и по возможности выделить, чему вы там научились. Да, даже на кассе в макдаке можно чему-то научиться: стрессоустойчивости, например. :)

Для айтишников: если вы проходите обучающие курсы на любом портале/по крутой книге (гуглите Learn %ваш язык программирования% - The Hard Way, например), и у вас есть наработанное портфолио учебных проектов/решённых задач, - смело укажите хотя бы пару-тройку месяцев этого опыта, дайте ссылку на сертификаты, гитхаб, публичный профиль на вашем любимом ресурсе для обучения, где показан прогресс. Главное не перестарайтесь: не указывайте год такого (на самом деле не практического) опыта, лучше укажите реальный срок обучения в описании, если считаете нужным.

Графа о себе

Подсказка: если HR будет печатать ваше резюме, допустим, с hh, сопроводительное письмо вместе с ним не распечатается. Так что неплохо было бы какие-то свои сильные стороны оставить и в «о себе».

-Вы переходите в новую область. Тут HR автоматически хочет от вас всех возможных объяснений такому поведению и всех комментариев, которые выделяют вас как специалиста в принципе, вне зависимости от области.

Насчёт объяснений не пугайтесь, если это IT, кратко объясните в резюме или сопроводительном, что в предыдущей области не пошло, а за компьютерами будущее. Мы видим такие резюме часто, мы привыкли и понимаем вас.

Что должно отсутствовать в резюме

1. Размытость, неточности. Каждый человек дорожит своим временем. Особенно рекрутер, у которого помимо вашего еще несколько десятков (а бывает даже и сотен) резюме. Никто не захочет тратить время на переспрашивание или уточнение у вас информации, которая важна для той или иной позиции, но при этом которую вы забыли либо не захотели указать.
2. Вода. Резюме — не сочинение на вольную тему и не автобиография, начиная с пеленок. Этот документ имеет четкую цель и предназначение, а значит пустословие в нем — дурной тон и заодно самый верный путь в корзину рекрутера.
3. Ложь. Также, не забывайте, что ложь всегда всплывает наружу — либо во время собеседования, либо непосредственно в ходе трудовых будней. Не выдавайте желаемое за действительное и составляйте честное резюме.

Если подытожить эти пункты, то верно следующее: хорошее резюме должно пестрить лаконичностью, быть честным и содержать только важную информацию.

Структура резюме

Фамилия, имя (First Name, Last Name)

Возраст (Age)

Контакты (Contacts) — город проживания, телефон, e-mail, Skype. (хорошо заполненный профиль в LinkedIn, ссылку на ваш профиль в GitHub). Скорее всего вас будут искать в соц-сетях так что преведите их в порядок.

Цель (Objective) — пункт, в котором вы сообщаете нанимателю, какую позицию желаете занимать в компании.

Резюме квалификации (Summary Of Qualification) — в этом разделе следует разместить основную информацию из резюме, которую будет полезно узнать работодателю в первую очередь (не более пяти ключевых тезисов, которые расскажут о вас и вашем опыте работы в выбранной сфере IT). Если не имеете каких-то сильных аргументов о вашем карьерном пути, пропустите этот пункт.

Профессиональные навыки (Hard Skills) — здесь вы описываете все технологии, которые знаете и умеете использовать. Рядом указывайте уровень владения каждой. Допустим, что-то знаете отлично, что-то базово, но готовы изучать дальше. Главное — пишите честно, ведь за каждую технологию или указанный вами язык программирования спросят.

Личные навыки (Soft Skills) — это нетехнические навыки, которыми вы обладаете. Умение работать в команде, решать конфликтные ситуации, быть внимательным к деталям, обладание хорошо развитым критическим мышлением, умение слушать и четко излагать свои мысли, а также другие подобные навыки относятся к софт скиллам. То есть, это те умения, которые помогают нам быть эффективными на работе. Много софт скиллов писать не надо, однако, минимум три наилучших своих качества указать необходимо.

Опыт (Experience) — следует указывать в обратном порядке, начиная с вашего последнего места работы. Каждое описываемое место работы должно иметь следующую структуру:

Структура резюме

Образование (Education) — информация о высшем образовании. Указывайте название ВУЗа, факультет, специальность, полученную степень, годы обучения.

Дополнительные навыки (Additional Skills) — информация про курсы, семинары, тренинги и другие обучающие программы, которые вы проходили самостоятельно. Тут же расскажите о ваших сертификатах, если они у вас имеются — здесь наступает их звездный час.

Знание языков (Knowledge Of Languages) — простой пункт. Составляется в виде “Язык — уровень владения языком”.

Дополнительная информация (Additional Information) — сведения о вас, которыми вы хотели бы поделиться: ваши интересы, увлечения, либо определенные особенности — наличие маленького ребенка, родственника, за которым вам необходимо присматривать и т. д. В конце концов, на работу берут не шестеренку, которая будет частью бездушного механизма, а личность с определенными знаниями, профессиональными навыками, взглядами на жизнь и чертами характера.

Общие советы

Требования к тексту. Для резюме выбирайте 12-14 кегль и шрифт Arial либо Times New Roman — текст будет максимально читабелен и без раздражающих элементов. Для новичков оптимальный размер текста — до одной страницы А4, от опытных программистов ожидают текста побольше, но лучше не превышать 2-х страниц. Причем большую часть должны занимать ваши скиллы и опыт работы.

Формат электронного резюме. На рассмотрение рекрутеру необходимо присылать резюме в формате PDF. Это самый универсальный формат электронных документов, который бесппроблемно открывается и читается на всех возможных устройствах.

Резюме на английском. Если вакансия составлена на английском языке, значит, работодатель однозначно хочет от кандидата англоязычное резюме. Даже если вакансия русскоязычная, а вы составили и прислали резюме на инглише, это будет солидным плюсом, ведь, как мы говорили ранее, данный язык в IT очень важен. Главное, не забывайте о грамотном написании. После составления резюме дайте его на проверку тому, кто хорошо знает английский.

Сопроводительное письмо

Сопроводительное письмо. В РБ практика написания сопроводительного письма не особо популярна, однако, правильное его составление может дать вам дополнительное преимущество перед другими кандидатами-конкурентами. Это письмо, которое дополняет резюме. Его цель — заинтересовать рекрутера/работодателя и создать хорошее первое впечатление о себе. Структура сопроводительного письма следующая:

1. Приветствие, обращение к работодателю.
2. Источник информации о вакансии.
3. Сведения, которые характеризуют вас как специалиста.
4. Демонстрация заинтересованности в вакансии.
5. Контактные данные.

Этапы собеседования

1. Собеседование с HR
2. Тестовое задание
3. Ревью тестового задания
4. Техническое собеседование

Как подготовиться к собеседованию?

1. Писать код на листочке 😊
2. Попытка зазубрить ответ
3. Решать задачи в уме
4. Отказ от решения\ответа на вопрос

Пару советов по поиску работы

1. Добавить 1к+ контактов в linkedin
2. Описать свой опыт и скиллы в профиле
3. Просматривать ленту и искать подходящие вакансии
4. Попросить товарища пошарить резюме в компании\в профиле

Проверка на софт скиллы

На таких интервью мягкие навыки проверяются с помощью вопросов вида:

1. Расскажите мне о ситуации, в которой вы..
2. Поделитесь примером того, как вы...
3. Представьте, что вы оказались в ситуации, где вам нужно...
4. Вспомните, как вы действовали в ситуации, когда...

Общие вопросы

1. Почему вы хотите сменить работу?
2. Как выбрали нашу компанию?
3. Какое направление вас больше всего интересует?

Вопросы по работе в команде

1. Что такое agile методология?
2. В чем отличие Kanban от Scrum?
3. Какие инструменты для работы в команде вы знаете?

Работа с заказчиком

1. Приведите пример того, как вы справились с неудовлетворенным клиентом.
2. Как бы вы поступили с клиентом, который, по вашему мнению, начал придираться?
3. Какие шаги вы предпринимаете, чтобы завоевать доверие клиента?

Рабочие вопросы

1. Вы склонны работать сверхурочно?
2. Приведите мне пример, когда вы сталкивались с этической дилеммой на работе.
3. Вы выполнили свою работу раньше, чем ожидали. Вы позволяете себе свободный день или собираетесь попросить больше заданий?

Стрессоустойчивость

1. Какие у вас методы борьбы со стрессом?
2. Какие вы используете хорошие способы для предотвращения стресса?
3. Какие рабочие ситуации вызывают у вас наибольшее напряжение?

Принятие решений

1. Вам нравится ответственность за принятие решений или вы бы предпочли оставить это кому-то другому?
2. Приведите пример, когда вам приходилось принимать решение под давлением. Как вы с этим справились?
3. Что вы делаете, если понимаете, что приняли плохое или неправильное решение?

Интервью

Сервис для прохождения интервью:

<https://www.pramp.com/>